

1. ElevationStableToken.sol

Purpose: A stablecoin minted 1:1 from verified revenue, tracked via IPFS receipts.

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title ElevationStableToken (EST)
 * @dev Pegged stable token backed 1:1 by verified revenue.
 * Minted by DAO-approved minters and tracked with IPFS receipts.
 */
contract ElevationStableToken is ERC20Burnable, AccessControl {
    bytes32 public
constant MINTER_ROLE =
keccak256("MINTER_ROLE");
    string public receiptBaseUri;

    struct MintReceipt {
        address to;
        uint256 amount;
        string ipfsHash;
        uint256 timestamp;
    }

    MintReceipt[] public mintReceipts;

    event Minted(address indexed to, uint256 amount, string ipfsHash);
    event BaseUriUpdated(string newBase);

    constructor() ERC20("Elevation Stable Token", "EST") {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function mint(address to, uint256 amount, string memory ipfsHash)
external onlyRole(MINTER_ROLE) {
    _mint(to, amount);
    mintReceipts.push(MintReceipt({
        to: to,
        amount: amount,
        ipfsHash: ipfsHash,
        timestamp: block.timestamp
    }));
    emit Minted(to, amount, ipfsHash);
}

    function setBaseUri(string calldata uri) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    receiptBaseUri = uri;
    emit BaseUriUpdated(uri);
}

    function getReceipts() external view
returns (MintReceipt[] memory) {
    return mintReceipts;
}

    function getReceiptLink(uint256 index) external view returns (string
memory) {
        require(index <
mintReceipts.length, "Invalid index");
        return string(abi.encodePacked(receiptBaseUri,
mintReceipts[index].ipfsHash));
    }
}
```

